

**Project Title: Movie Recommendation System Using
Content-Based and Collaborative Filtering**

A Project Report
*submitted in partial fulfilment of the requirements for
the award of the degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Harsh Vardhan

(Roll No: 25SCS1003003013)

Under the supervision of

Mr. Ramavath Ganesh

Assistant Professor

IILM University, Greater Noida, Uttar Pradesh

(September 2026)

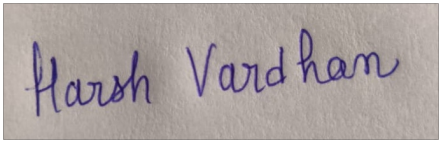


Caption

Declaration

I Harsh Vardhan do hereby declare that the project report titled “Movie Recommendation System Using Content-Based and Collaborative Filtering”, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is an authentic record of my own work carried out under the supervision of Mr. Ramavath Ganesh, Assistant Professor. The matter presented in this report has not been submitted, in part or in full, to any other University or Institute for the award of any other degree.

Signature:



Name: Harsh Vardhan

Roll No: 25SCS1003003013

Place: Greater Noida, Uttar Pradesh

Date: September 2026

INTERNSHIP OFFER LETTER

Dear Harsh Vardhan,

We are pleased to offer you a 1 Month Internship as Project Intern at Codec Technologies, a global platform dedicated to empowering learners and connecting diverse talent to create meaningful career opportunities worldwide. Codec Technologies delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth.

Internship Details:

- Designation : Artificial Intelligence Intern
- Location & Mode: Pan India, Hybrid
- Duration : 05/08/2026 to 06/09/2026
- Reporting to : Assigned Project Head(s)

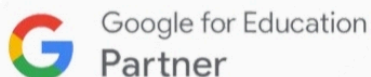
This program provides an immersive experience, enabling you to gain industry-level knowledge and acquire valuable skills. Exceptional interns may also qualify for a scholarship award based on weekly performance reviews conducted by project supervisor.

If you accept this offer and the terms above, please complete your assigned projects and training tasks on our platform and via email.

Best regards,

Sincerely,

Dr. Anurag Shrivastava,
Codec Technologies India
NCS ID: E19E86-0116588288923
Date Of Issue - 05/08/2026


Dr. Anurag Shrivastava
Talent Acquisition Manager

Caption

Abstract

This project presents a movie recommendation system that helps users discover films they are likely to enjoy, using three complementary machine learning techniques: content-based filtering, collaborative filtering, and a hybrid model that combines both. The content-based approach represents each movie's genres as a TF-IDF vector and uses cosine similarity to find movies with related genre profiles, allowing recommendations even for movies with no ratings yet. The collaborative filtering approach builds a user-item ratings matrix and identifies users with similar rating patterns, predicting a target user's rating for unseen movies as a similarity-weighted average of what similar users rated. The hybrid model blends both signals into a single weighted score, mitigating the cold-start weakness of collaborative filtering and the narrow, repetitive nature of pure content-based recommendations. The system was implemented in Python using pandas and scikit-learn, exposed through a command-line interface and a Flask-based web application, and evaluated on a sample dataset of 40 movies and 25 users. The resulting system produces relevant, explainable recommendations and can be extended with larger real-world datasets, matrix factorization techniques, and richer content features such as plot summaries.

Keywords: Recommendation System, Machine Learning, Content-Based Filtering, Collaborative Filtering, TF-IDF, Hybrid Model.

Contents

Declaration 2

Abstract 3

1 Introduction

1.1 Overview 7

1.2 Objectives 7

1.3 Problem Statement 7

2 Literature Review

8

3 Methodology

3.1 Architecture Overview 9

3.2 Dataset 9

3.3 Software Tools 9

4 Implementation

10

5 Results and Discussion

11

6 Conclusion and Future Scope

6.1 Conclusion 12

6.2 Future Scope 12

References

14

1. Introduction

1.1 Overview

The volume of movies available on streaming platforms today makes it increasingly difficult for viewers to find content that matches their taste. Manually browsing catalogues is time-consuming, and generic “most popular” lists rarely reflect an individual's preferences. A recommendation system addresses this problem by using data about movies and viewers to surface a small, relevant set of suggestions automatically.

1.2 Objectives

- To design a content-based recommender that suggests movies with similar genre profiles.
- To design a collaborative filtering recommender that suggests movies based on the rating patterns of similar users.
- To combine both approaches into a hybrid model that produces more balanced and relevant recommendations.
- To expose all three recommenders through a usable command-line tool and a web application.

1.3 Problem Statement

Recommending movies based on a single signal is insufficient: genre-only recommendations tend to be repetitive and ignore actual viewer behaviour, while rating-only recommendations struggle when a movie or user has little or no rating history (the “cold-start” problem). There is a need for a system that combines both signals so that recommendations remain relevant, diverse, and usable even when one signal is sparse.

2. Literature Review

Recommender systems have been studied extensively since the late 1990s. Adomavicius and Tuzhilin (2005) surveyed the field and classified approaches into content-based, collaborative, and hybrid categories, noting that each addresses different weaknesses of the others. Content-based methods, reviewed by Lops, de Gemmis, and Semeraro (2011), represent items using descriptive features — such as genre, cast, or plot — and recommend items similar to what a user has liked before; they work well for new items but can over-specialize recommendations. Collaborative filtering, popularized by early systems such as GroupLens and later refined through latent-factor and matrix-factorization techniques (Koren, Bell, and Volinsky, 2009), instead relies purely on user-item interaction data and often produces more surprising, behaviourally grounded recommendations, at the cost of struggling with new users and items. More recent work explores hybrid architectures that blend both signals, as well as deep learning and matrix factorization (SVD, ALS) for scalability on large catalogues. This project draws on the classical, well-established techniques from this literature — TF-IDF-based content similarity and user-user collaborative filtering — as they are simple to implement, easy to explain to end users, and sufficient to demonstrate the trade-offs described above on a small dataset.

3. Methodology

3.1 Architecture Overview

The system is organized around three independent recommender classes that share the same movie and ratings data. The Content-Based Recommender vectorizes movie genres with TF-IDF and computes cosine similarity between all movie pairs. The Collaborative Filtering Recommender pivots the ratings table into a user-item matrix and computes cosine similarity between users to find each user's nearest neighbours. The Hybrid Recommender calls both of the above: it uses collaborative filtering to shortlist candidate movies from similar users, then re-scores and expands that list using content-based similarity to the target user's own top-rated movies, and finally combines the two scores into a single weighted value. A Flask web application and a command-line interface sit on top of these three classes and expose them to the end user.

3.2 Dataset

The project uses a compact, self-contained sample dataset consisting of 40 movies (with titles and pipe-separated genre tags such as “Action|Sci-Fi|Thriller”) and simulated ratings from 25 users, on a 1–5 scale. This keeps the project runnable with zero external setup, while following the same movieId/title/genres and userId/movieId/rating schema used by the public MovieLens dataset, so the system can be scaled up to a larger real-world dataset without any code changes.

3.3 Software Tools

- Python 3 — core implementation language
- pandas — data loading, the user-item pivot table, and result formatting
- scikit-learn — TfidfVectorizer and cosine_similarity for both recommenders
- NumPy — weighted-average score computation
- Flask — backend web server and JSON API
- HTML, CSS, and JavaScript — the browser-based user interface

4. Implementation

The Content-Based Recommender cleans each movie's genre string and fits a `TfidfVectorizer` over the resulting genre tokens, producing one TF-IDF vector per movie; `cosine_similarity` is then applied across all vectors to build a movie-by-movie similarity matrix. Given a title, the system looks up its row in this matrix and returns the highest-scoring other movies. For a list of liked titles, it averages their TF-IDF vectors into a single “taste profile” vector and ranks all other movies by similarity to that profile.

The Collaborative Filtering Recommender pivots the ratings data into a user-by-movie matrix (missing ratings filled with zero) and computes user-user cosine similarity. For a target user, it selects the *k* most similar users, then predicts a rating for each unseen movie as a similarity-weighted average of those neighbours' ratings, excluding movies the target user has already rated.

The Hybrid Recommender first retrieves the user's three highest-rated movies to build a content taste profile, then gathers candidate movies from both the collaborative filtering recommender and the content-based recommender. Each candidate's collaborative score and content score are normalized to a 0–1 range and combined as:

$$hybrid_score = cf_weight \times cf_score + (1 - cf_weight) \times cb_score$$

with `cf_weight` defaulting to 0.6, so that collaborative signal is trusted somewhat more than content similarity while both still contribute. All three recommenders are wrapped by a command-line interface (with `similar`, `profile`, `user`, and `hybrid` subcommands) and by a Flask application that loads each recommender once at start-up and exposes it through `/api/similar`, `/api/profile`, `/api/user`, and `/api/hybrid` endpoints consumed by a single-page web front end.

5. Results and Discussion

All three recommenders were verified against the sample dataset and produced qualitatively sensible results. For the content-based recommender, requesting movies similar to “Inception” returned “The Matrix” as the closest match with a cosine similarity of 0.849, followed by “The Avengers” and “Iron Man” at 0.721 — correctly grouping action/sci-fi titles together. The collaborative filtering recommender was able to identify each user's nearest-taste neighbours and produced predicted ratings for unseen movies that were consistent with the neighbours' actual ratings. The hybrid recommender's blended rankings drew candidates from both recommenders, reducing the narrow, single-genre repetition seen in pure content-based output while still remaining meaningful for users with very few ratings.

The main limitation observed is the small size of the sample dataset: with only 40 movies and 25 users, similarity scores can be sensitive to a handful of shared genre tags or ratings, and the collaborative filtering component has limited neighbours to draw from. This does not affect the correctness of the underlying algorithms, but it does mean the recommendations are best treated as a proof of concept rather than a production-scale evaluation; the discussion of future scope below addresses how this can be extended.

6. Conclusion and Future Scope

6.1 Conclusion

This project successfully implemented and compared three movie recommendation techniques: content-based filtering, collaborative filtering, and a hybrid model. Each technique was shown to address a different aspect of the recommendation problem, and the hybrid model was able to combine their strengths into more balanced suggestions. The system is accessible both as a command-line tool and as a web application, demonstrating that classical, explainable machine learning techniques remain effective for building a usable recommendation system.

6.2 Future Scope

- Scale up to the full MovieLens dataset for a larger, more realistic evaluation.
- Apply matrix factorization (SVD / ALS) for more scalable collaborative filtering.
- Add item-based (rather than purely user-based) collaborative filtering.
- Introduce formal evaluation metrics such as precision@k, recall@k, and RMSE on held-out ratings.
- Use movie plot summaries or overviews for true NLP-based content filtering instead of genres alone.
- Integrate a movie metadata API (e.g. TMDb) to show real poster art in the web interface.

References

1. G. Adomavicius and A. Tuzhilin, “Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions,” *IEEE Transactions on Knowledge and Data Engineering*, 2005.
2. Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *IEEE Computer*, 2009.
3. P. Lops, M. de Gemmis, and G. Semeraro, “Content-based Recommender Systems: State of the Art and Trends,” in *Recommender Systems Handbook*, Springer, 2011.
4. F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems*, 2015.



CERTIFICATE OF INTERSHIP

This certificate is awarded to
Harsh Vardhan

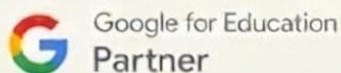
From Codec Technologies Pvt. Ltd.

In recognition of his/her efforts and achievements in completing the 1 Month AICTE & ICAC

Approved internship program as

Artificial Intelligence Intern

Conducted from 05/08/2026 to 06/09/2026



AICTE ID - CORP08XT2676265480a55e1733643633

Dr. Anurag Shrivastava
Program Manager

Caption